

MPDW Project Report — Phase 2

David Castro^{1,2}, Yaroslav Hayduk^{1,3}, and Bruno Baptista^{1,4}

¹ FCT UNL, Department of Computer Science

² 60973, djc.castro@campus.fct.unl.pt

³ 60739, y.hayduk@campus.fct.unl.pt

⁴ 59815, bm.baptista@campus.fct.unl.pt

1 Introduction

This project addresses the challenge of semantic video moment retrieval using transformer-based architectures. The goal is to create a system that can understand user queries and retrieve temporally relevant segments (“video moments”) from long videos.

In phase 1, we focused on understanding and building embedding spaces, indexing video captions, and enabling semantic search using dual encoders and OpenSearch. This involved parsing the ActivityNet Captions dataset (used as our base for video content and metadata), selecting key videos and processing their moments, extracting representative keyframes, computing both textual and visual embeddings, and indexing and querying the data using OpenSearch with support for k-nearest-neighbor search.

In Phase 2, we extended this foundation to support cross-modal retrieval and multimodal reasoning. By embedding both captions and keyframes into a shared representation space using CLIP, we enabled text-to-image, image-to-text, and same-modality semantic retrieval. Additionally, we integrated LLaVA, a large vision-language model, to support visual question answering (VQA). In this setup, the system retrieves a relevant video frame for a given visual question and uses LLaVA to generate a natural language response based on the content of that frame.

These additions allow the system to move beyond simple retrieval and support more complex, open-ended interaction with video content through both cross-modal search and generative question answering.

To gain deeper insight into model behavior, we also explored CLIP interpretability by visualizing similarity scores and attention heatmaps between captions and video frames.

2 System Architecture

Our Phase 2 system forms a five-stage pipeline that converts raw ActivityNet videos into an indexable, query-able, and interpretable LVLM service.

1. **Pre-processing**
 - Trim videos to annotated moments and extract key-frames.
 - Merge timestamped captions from `val_1.json` and `val_2.json`.
2. **Multimodal Embedding Generation**
 - *Sentence-BERT MPNet* $\rightarrow$ 768-d `caption_vec` (semantic text search).
 - *CLIP ViT-B/32 – text encoder* $\rightarrow$ 512-d `visual_caption_vec`.
 - *CLIP ViT-B/32 – image encoder* $\rightarrow$ 512-d `keyframe_vec`.
3. **Indexing (OpenSearch)** Three `knn_vector` fields, HNSW (M=16, ef=256), plus metadata (timestamps, resolution).
4. **Cross-modal Retrieval API** `search(query, modality)` dispatches to one of *text $\rightarrow$ image*, *image $\rightarrow$ text*, *image $\rightarrow$ image* and returns top-*k* ranked hits.
5. **Retrieval-Augmented VQA** The best key-frame is fed, together with the user question, to *LLaVA-Phi-3*. Answers and CLIP similarities are cached and logged.
6. **Interpretability Module** Generates (i) temporal similarity curves, (ii) Grad-CAM heat-maps for 10 frames, and (iii) a caption $\times$ frame contrastive matrix.

2.1 Embedding Representations and Index Schema

We moved from a *dual* to a **tri-encoder** design so that the text and image branches share the same CLIP space. Three independent vector fields are written to OpenSearch (Table 2.1).

- **Sentence-BERT (MPNet)** Captions $\rightarrow$ 768-d `caption_vec`. Used for classic semantic *text $\rightarrow$ text* retrieval baselines.
- **CLIP ViT-B/32 — text encoder** Captions $\rightarrow$ 512-d `visual_caption_vec`. Aligns captions with images and enables *text $\rightarrow$ image* queries.
- **CLIP ViT-B/32 — image encoder** Key-frames $\rightarrow$ 512-d `keyframe_vec`. Supports *image $\rightarrow$ image* and *image $\rightarrow$ text*.

Field	Model	Dim.
<code>caption_vec</code>	SBERT MPNet	768
<code>visual_caption_vec</code>	CLIP (text)	512
<code>keyframe_vec</code>	CLIP (image)	512

Document granularity. One index document corresponds to a (*key-frame*, *caption*) pair. A key-frame that overlaps two temporal moments is therefore duplicated, each time with the relevant caption and its own ID. Metadata stored alongside the vectors now includes: `video_id`, `start_timestamp`,

`end_timestamp`, `duration`, `resolution`, and the local `keyframe_path`. All vectors are ℓ_2 -normalised before insertion so cosine similarity reduces to the inner-product HNSW metric configured in OpenSearch.

Contextual embeddings: For visualizing contextual embeddings, we utilized the caption of one of our videos: “A man is in a skateboard track, then he throws a bowling ball that goes around and hits the pins”. The resulting graphs of the layers zero to eleven can be seen in Fig. 1.

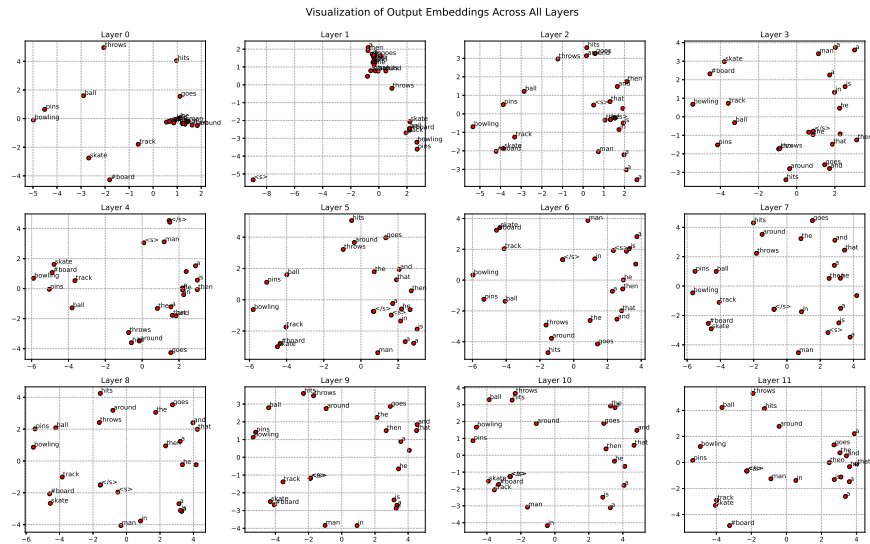


Fig. 1. Contextual embeddings

Surprisingly, in the first layer, the embeddings are very similar to their values in the eleventh layer, but this outcome is merely coincidental, while in the second layer we get something that would be more expected, that is, most embeddings are fairly close together with seemingly no relation or logic. With the progression of the layers, we can see the embeddings disperse and form sensible groups. Namely, we can see the terms “bowling” and “pins” very close together and far from most other words. This makes sense given how specific and relatively rare those words are. The same can be observed for skate, track and board — the suffix of skateboard, hence the proximity. Lastly, the most obvious cluster, on the right: the most common english words. Words such as “a”, “the”, “that” and “and” which, due to the frequent and indiscriminate use that is given to them, hold very little semantic value and weight in the meaning of sentences.

Positional embeddings: By repeating the previous exercise with a sentence of the same word repeated 20 times instead, what we get is the positional embeddings of our sentence (see Fig. 2). The result is a phenomenon that can be explained by the gradual decrease of positional importance throughout the sentence. Stated in simpler terms, the farther a token is from the first word, the smaller the importance of its position. The first word is the one we can see down at the bottom, the farthest from all the others, as its position has the highest relevance. As we move forth in the sentence, the position relevance gradually decreases and we get increasingly similar embeddings. Due to this, we see the converging effect that results in the cluster of points we see at the top, with all the embeddings of the last words close together.

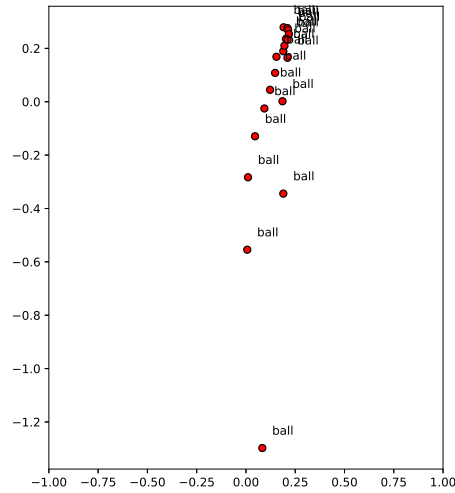


Fig. 2. Positional embeddings

2.2 Large Vision and Language Models

While Phase 2 delves deeper into large vision and language models, in Phase 1 we already used pretrained transformers:

- **MPNet (All-MPNet-Base-V2)** for generating context-aware text embeddings.
- **CLIP (ViT-B/32)** to embed keyframes into a multimodal representation space.

In Phase 2, we extended the use of CLIP to achieve full **cross-modal retrieval** functionality by embedding both keyframes and captions using CLIP’s respective image and text encoders. This allows us to represent both modalities in the same embedding space, enabling:

- **Text-to-image retrieval**, where a natural language query retrieves relevant keyframes.
- **Image-to-text retrieval**, where a visual frame can retrieve semantically aligned captions.
- **Image-to-image** and **text-to-text retrieval**, leveraging the shared embedding space for similarity search across same-modality inputs.

All CLIP embeddings, whether from keyframes or captions, were indexed in OpenSearch, allowing for real-time similarity search.

We also integrated **LLaVA (Large Language and Vision Assistant)** for visual question answering (VQA). LLaVA combines CLIP’s visual encoder with a language model fine-tuned for multimodal reasoning.

In our system, we used retrieval-augmented VQA:

1. A visual question is submitted.
2. The embedding for the question is computed using CLIP’s text encoder.
3. A k-NN search is performed to retrieve the most relevant keyframe based on semantic similarity in the shared embedding space.
4. Both the retrieved frame and the question are passed to LLaVA.
5. LLaVA returns a grounded, natural language answer to the question based on the content of the frame.

Below are a couple of examples of the usage of LLaVa:

Question 1

Question: *"What is the man doing at the skatepark?"*

Retrieved Frame: Fig. 3

Generated answer: The image shows a group of people, including a young person and others, gathered around a skateboard ramp in an outdoor skate park. One boy with a skateboard is positioned on top of a half-pipe, seemingly preparing to ride down it or possibly just finished his run. The man standing by the side of the ramp could be observing, watching the skateboarder, providing assistance, or waiting for his turn. There are also several chairs visible in the

area, which might provide a place for spectators to sit and watch the activities at the skate park.

Question 2

Question: *"What color are the boxing gloves?"*

Retrieved Frame: Fig. 4

Generated answer: The boxing gloves in the image are red.

Question 3

Question: *"Where is the man playing bowling?"*

Retrieved Frame: Fig. 5

Generated answer: Inside a building, specifically in an indoor alley.



Fig. 3. Retrieved frame for question "What is the man doing at the skatepark?"



Fig. 4. Retrieved frame for question "What color are the boxing gloves?"



Fig. 5. Retrieved frame for question "Where is the man bowling?"

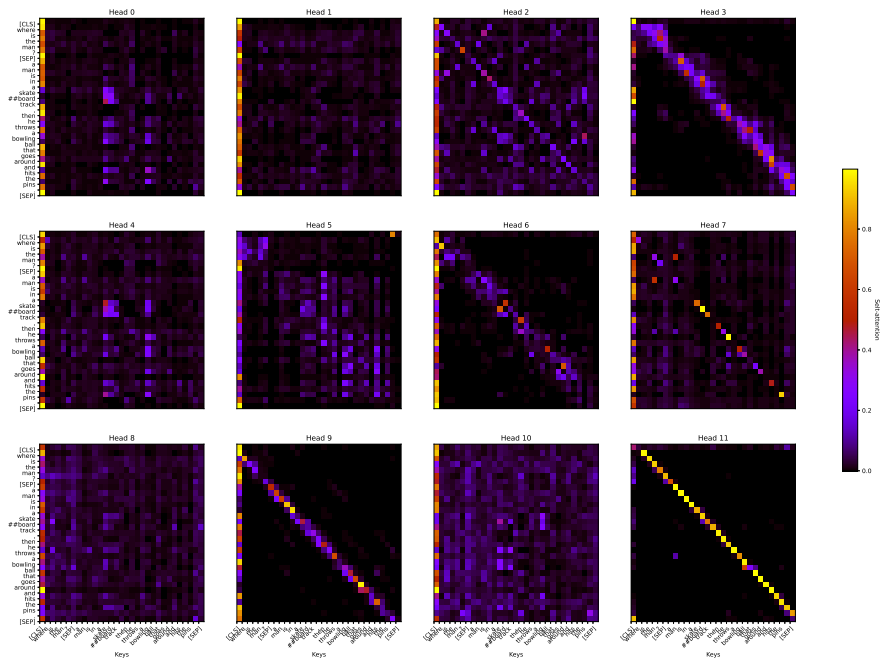
2.3 Attention

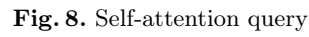
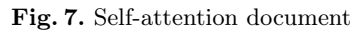
Self-attention for Cross-Encoder: To analyze the model’s self-attention mechanism we generated every head’s self-attention for the input “where is the man? A man is in a skateboard track, then he throws a bowling ball that goes around and hits the pins”. When looking at the different heads there are multiple patterns we can identify, giving us an intuition of their purpose. A lot of them also simply pay a general, overall attention to the whole sentence. If we look at figure 6, head 2 looks at every token relatively the same, with a special interest in itself, given the distinctive diagonal. This phenomenon can be seen even more clearly in head 7. Head 3 shows us a very clear diagonal pattern, where each token pays higher attention to itself and the around 3 to 6 right next to it, with virtually no attention to all the others; this could be loosely interpreted as: this head solely pays attention to the current token and the few right next to it. Similar effects can be seen in other heads, such as head 6, 9 and 11. Head 6 seems to be paying most of its attention to the couple of tokens right before and right after the current token. Head 9 attends pretty much only to the two tokens right behind the current token. While finally, head 11, pays attention almost exclusively to the token immediately after.

Self-Attention for Dual-Encoder: In the dual-encoder architecture, the query and document are processed independently, which results in separate self-attention mechanisms for each. To analyze this, we visualized the attention heads for both the query sequence “Where is the man?” and the document “A man is in a skateboard track, then he throws a bowling ball that goes around and hits the pins”, as shown in Figures 7 and 8.

For the query, attention patterns are more localized and interpretable due to its short length. Many heads show weak diagonals, indicating that each token attends mostly to those ahead of it. In contrast, the document’s self-attention shows more diverse behavior. Several heads focus attention across broader spans of the input, capturing syntactic structures or semantic relations within the sentence. For instance, tokens such as “*skate*” or “*hits*” often receive and distribute attention across related nouns like “*board*” or “*pins*”, suggesting an awareness of action-object relationships. Other heads demonstrate local attention, where each token mainly attends to subsequent neighbors.

Unlike the cross-encoder, the dual-encoder architecture does not allow for query-document token interaction within the attention layers. Instead, it relies on the resulting embeddings of the entire sequences to compute similarity. We can clearly see from our results how similar their embeddings are, which greatly enhances that operation’s precision.

**Fig. 6.** Self-attention



2.4 CLIP Interpretability

In this section, we interpret the embeddings generated by our CLIP model for both images and text, and the relation they display with each other, as well as analyze its attention mechanism and a representation of it.

Language-Vision temporal similarity To analyze the temporal similarity between a caption and a video, we selected a video along with all its frames, chose a caption, and computed the similarity between its CLIP embedding and the embeddings of each frame in the video. For our particular example the caption was '*Two men interview each other on brown chairs*', belonging to a segment of a video by *Dude Perfect*, where they perform multiple bowling ball shenanigans, with the results shown in Fig.9. In order to obtain a smoother graph, we averaged each point with the one immediately before and immediately after. In this figure, we are able to see the sections where the similarity score is highest, and where is lowest (apart from the beginning and ending), along with a small thumbnail of the frame for ease of analysis. As we can clearly observe, the similarity score is fairly low throughout the video, as most of it features said bowling shenanigans; but when it cuts to the interview section, the one that exactly corresponds to the two men interviewing each other, it spikes up, resulting in the effect we see. Worth noting is the other big spike we see before that, corresponding also to two men interviewing each other, although not on brown chairs. The lowest point we see features neither men interviewing each other nor brown chairs, accounting for the low similarity score. The particular spike down, in comparison to other moments, can be explained by the fact that throughout the video, in short segments are featured the *dudes* from *dude perfect* briefly speaking to each other - possibly being interpreted as some form of *two men interviewing each other* -, while this part, around frame 55, is mostly just bowling a bowling ball trickshot.

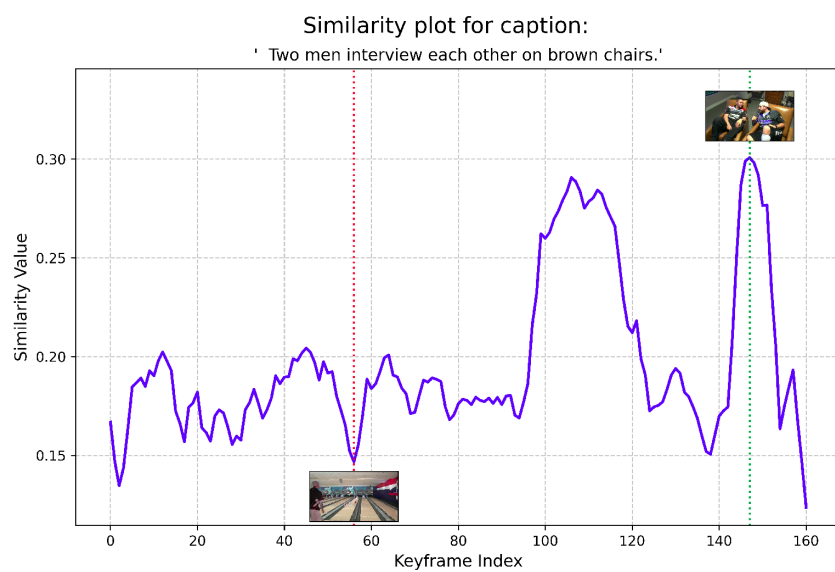


Fig. 9. Caption-Keyframe similarity score example 1

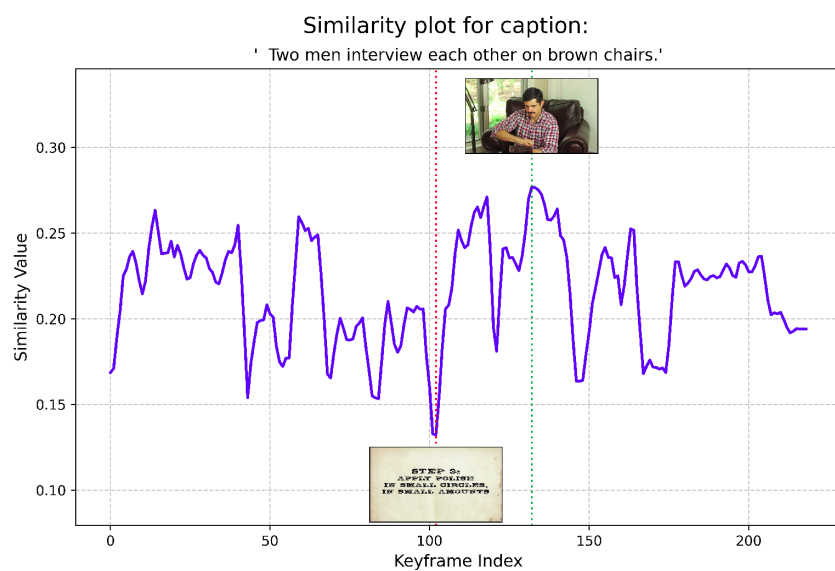


Fig. 10. Caption-Keyframe similarity score example 2

By taking this same caption and comparing it with another video, we are able to see that, when something that could be vaguely described with it happens, its similarity score increases, as seen in Fig.10. The other video we used for this comparison is about how to shine shoes - a very different topic from bowling ball trickshots. But when we look at the graph - and particularly at the segment with the highest score - in the keyframe corresponding to that moment we can see something that could be considered as very similar to our caption: a man talking in a black chair. As for the moment with the lowest similarity, it could be anything really, but in our case it was one of the transitions, where the frame shows only text.

Language-Vision contrastive moments Now to give a broader view, we analyze a heatmap of the CLIP embeddings corresponding to every segment’s caption of the show shining video and a frame extracted exactly from the middle of it. The results are shown in Fig.11. Due to the method we used of extracting the frame in the exact middle and the fact that some segments are very long, whereas the caption refers to a only a short part, the results are exactly perfect, however, a fairly strong match can still be seen along the main diagonal, with much darker colors on the counter-diagonal.

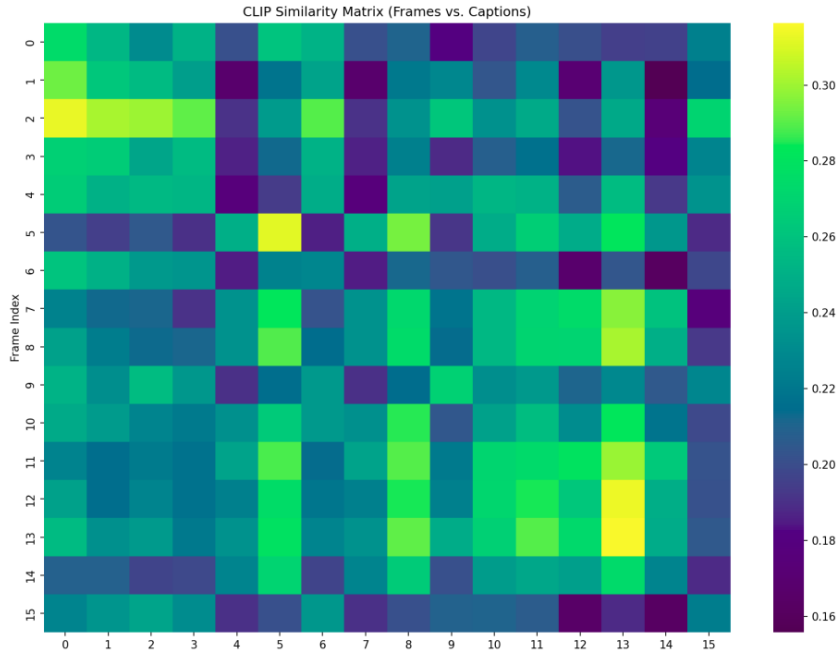


Fig. 11. Caption-Keyframe similarity heatmap

Finally, to really visualize the attention that is being given to certain parts of a frame, given a caption, we adapted the provided interpretable visual maps to generate a matrix of visualizations for the same frames and captions used above,

seen in Fig.12. These representations really nail the relationship between what the text describes and what is the image. In particular, if we look at the image resulting of the caption '*The man shines his shoes*' with the corresponding frame for that moment, it is very clear how the attention is being directed at the part of the image where the shoes are, whereas for the same frame, with the caption '*A man is standing inside a bathroom*', the focus shifts upwards - with the hotter areas being the parts of the frame where we can see the man.

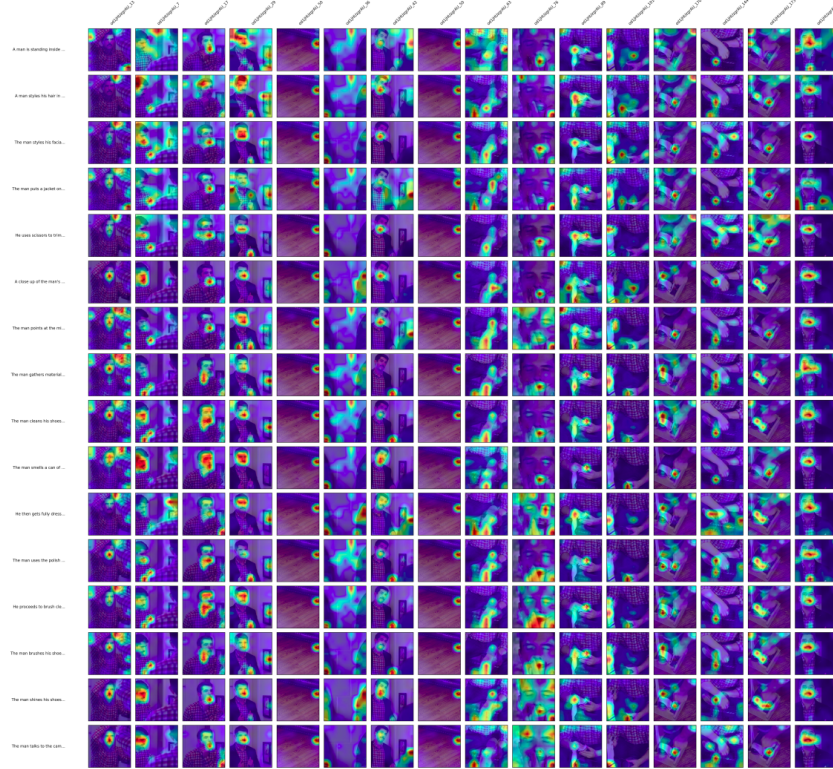


Fig. 12. Attention heatmap matrix

3 Evaluation

3.1 Dataset Description

We used the ActivityNet Captions dataset, which provides:

- ~20k YouTube videos with natural language descriptions of events,
- Timestamp-aligned captions for video moments.

In our implementation, we selected from the dataset 10 videos with a reasonable amount of moments. For each:

- Keyframes were extracted using PyAV by saving either keyframes or one frame per second.
- Corresponding captions were merged and aligned using metadata from three different files: `val_1.json`, `val_2.json`, and `activity_net.v1-3.min.json`.

The resulting data was indexed in OpenSearch using custom mappings to support both text-based and embedding-based (knn) search.

4 Phase 3: Visualizing text with keyframes

4.1 Our idea

When reading a book, say a piece of fiction or romance, as the author paints a picture of the different sceneries and characters, we try to imagine them in our head. As we read, each different word steers our imagination in a slightly different direction, until a full picture is painted. When using a CLIP model to encode a caption, the model is attempting to generate an embedding that is able to encapsulate the meaning and context of that piece of text. Considering this, we could make an analogy between these embeddings and the full picture we paint in our head when reading a book. For this phase, what we attempted was to develop an approach that would allow us to simulate this mechanism, where we picture an image in our mind after every word we read.

4.2 Our approach

Simulating this process could be done in many different ways, but with the goal of demonstrating our understanding of the course’s topics and technologies, we decided to make the most out of what we had already done and reutilize what we had already implemented within this project. The desired result was a sequence of images that could represent what we imagine in our brain as we read a sequence of text. For obvious reasons we had very moderate expectations of the realism of the result, especially since we were only using the frames of the videos we indexed, but we employed the tools at our disposal to make it as good as possible.

Implemented solution — As we have established, what we want is to generate a video (or, more precisely, a sequence of images) given a textual input. For this, we used embeddings generated from that input to query our OpenSearch index for the most similar frames and stitched them all together. Simply using the final generated embedding of our CLIP model for an input would not be enough, as this embedding would not capture the nuance in meaning of each word separately and in sequence. To capture that nuance we would need an embedding for each of our words — or tokens.

In an initial attempt we tried to just split our input in words and encode each of these words separately, but we quickly realized this would not be correct, as the generated embeddings for each word would completely lack any nuance and context of how that word was used. Having concluded this, in the next iteration we decided to pass the input whole and tried to extract the embedding of each token after the final layer, in an attempt to get their contextualized embeddings. When analyzing the result, what we found was that most frames were very similar, if not equal, and did not match in any way the meaning of the word they were representing. It was at this point that we finally got a good grasp of how attention works and how that translates to the embeddings the model computes. By the final layer, the embedding of each token had converged to similar representations of the overall meaning of our input, so we realized this was also not what we wanted.

On the third iteration we finally realized what we needed was the outputted embeddings of the first layer. By this point the model has computed the attention of all of our tokens and updated their embeddings once, resulting in these embeddings being contextualized considering the whole input, but not yet having their individual meanings completely diluted. Using these embeddings, we query our index to get the frame most similar to each token and stitch them together.

Methods for improving the result — By running the previously described solution, what we get is a seemingly nonsensical slideshow of video frames. To simulate a more interesting video, we expanded our implementation to leverage some other tools we had worked with in this project.

Since we had indexed our frames with their ordering number, the first thing we did was use this to get more than one frame per token. To be precise, we fetched the next and previous frame, if they existed, of the most similar frame. Since one of these frames could be from a separate section of the video, applying this blindly could be useless, or even worsen the video. To mitigate this, we also calculate the similarity between the main frame — the one we get from querying OpenSearch with a token’s embedding — and the previous and next frames. If their similarities are lower than a threshold we defined, we discard them.

This technique allowed for less jumping around but, from one word to the next, the frames are still generally very different, resulting in very noticeable transitions. With this in mind, we used something that had not yet been explored in the context of this project: a video generating model. Each token in our input translates to a scene of 1 to 3 frames in the generated video, so, by feeding the last frame corresponding to a token and the first frame corresponding to the next token to this generative model, we get a much smoother transition between scenes. Worth noting that the model we used was fairly small, so it was not capable of generating particularly detailed or high quality images, but we

considered this a feature, not a bug, since it resulted in the dream-like effect we were aiming for since the beginning.